

GUIDE COMPLET DE L'APPLICATION

Explication Fichier par Fichier & Architecture Codebase — Soutenance Technique

Ce document couvre toute l'application : chaque fichier, chaque fonction, chaque page.

Les explications sont en tunisien technique avec les termes exacts.

[FOLDER] ARBORESCENCE GLOBALE DU PROJET

banking-api-secure/	
■■■ app/	← [CORE] LE BACKEND (FastAPI / Python)
■■■ main.py	← Point d'entrée + Middlewares de sécurité
■■■ db.py	← Connexion MongoDB + Collections
■■■ rate_limiter.py	← Configuration SlowAPI (limitation de débit)
■■■ models/	← Modèles Pydantic (schémas de données)
■■■ user.py	
■■■ account.py	
■■■ routes/	← [ROUTE] TOUTES LES ROUTES API (13 fichiers)
■■■ auth.py	← Login, 2FA, Face ID, Mot de passe oublié
■■■ user.py	← Inscription, Profil, Paramètres, Biométrie
■■■ account.py	← Comptes, Carte, Virements, Paiements, Factures (LE PLUS GROS)
■■■ email_verification.py	← Envoi d'OTP par email (SMTP Gmail)
■■■ beneficiary.py	← Gestion des bénéficiaires + CoP
■■■ budget.py	← Budgets mensuels par catégorie
■■■ financial_hub.py	← Conseils IA (Gemini) + Rendez-vous
■■■ support.py	← Chatbot IA + Messagerie sécurisée
■■■ admin.py	← Dashboard Admin + SIEM + Blocage IP
■■■ alerts.py	← Flux SSE (alertes temps réel)
■■■ activity.py	← Journal d'activité utilisateur
■■■ payment.py	← Route de paiement simple (legacy)
■■■ locations.py	← Agences (deprecated, frontend direct)
■■■ security/	← [SECURITY] COUCHE SÉCURITÉ
■■■ auth.py	← JWT, Keycloak JWKS, Hachage, verify_token
■■■ logger.py	← Double logging (MongoDB + fichier JSONL)
■■■ utils/	← [SOAR/SIEM] MOTEUR SOAR / SIEM
■■■ threat_monitor.py	← Thread de surveillance + Score T_decay + Email
■■■ security_state.py	← Pare-feu applicatif (IP bloquées en mémoire)
■■■ alert_store.py	← Bus d'alertes thread-safe (deque)
■■■ security_analyzer.py	← Analyse heuristique + IA Gemini des logs
■■■ correlator.py	← Corrélation multi-sources (HTTP + MongoDB)
■■■ bank_frontend/	← [FRONTEND] LE FRONTEND (Flask + Jinja2 + HTML/CSS/JS)
■■■ app.py	← Serveur Flask (82 Ko !) – routing + proxy API
■■■ templates/	← 33 pages HTML
■■■ login.html / signup.html / verify_login.html	
■■■ dashboard.html / account_details.html / card_details.html	
■■■ budget.html / analytics.html / journal.html	
■■■ beneficiaries.html / bills.html / services.html	
■■■ settings.html / messages.html / advice.html	
■■■ admin_dashboard.html / admin_users.html / admin_accounts.html	

■ ■■■ ... (33 pages au total)

■■■ nginx.conf	← [INFRA] PASSERELLE NGINX (mTLS + Rate Limit + Headers)
■■■ docker-compose.yml	← ■ ORCHESTRATION (10 services, 4 réseaux)
■■■ Dockerfile.backend	← Image Python pour FastAPI
■■■ .env	← Variables secrètes (Gmail, Keycloak, Gemini)
■■■ elk/	← Configuration Filebeat + Logstash
■■■ nginx_certs/	← Certificats mTLS (CA, serveur, client)
■■■ scripts/	← Scripts de test (ZAP, performance, brute force)
■■■ security_audit.log	← Fichier de logs centralisé

[CORE] PARTIE 1 : LE BACKEND (Dossier `app/`)

[FILE] app/main.py — Le Chef d'Orchestre

Chnowa ya3mel : Houwa el point d'entrée mta3 l'API el koll. Ye3mel 3 7weyej fondamentaux :

1. Middleware de Sécurité Combiné (Lignes 78-118)

Kol requête HTTP li toussel → te3addi mel middleware hedha AVANT ma toussel l les routes

- Ligne 83-85 : Yextracti el IP réelle du client (X-Real-IP mel NGINX, mouch mel client directement)
- Ligne 88-89 : Ychouf ken el IP bloquée walla le : ken bloquée → [403 IP Blocked by SIEM](#)
- Lignes 95-98 : Yzid les Security Headers OWASP :
- **X-Content-Type-Options: nosniff** → Ye3dem el navigateur men yedevini el MIME type
- **X-Frame-Options: DENY** → Yemna3 el Clickjacking (iframe)
- **X-XSS-Protection: 1; mode=block** → Protection XSS
- **Strict-Transport-Security** (HSTS) → Yeforci HTTPS sena kamla
- Lignes 110-116 : Yloggui kol requête fi [security_audit.log](#) (IP, Method, Path, Status, Duration)

2. Configuration CORS (Ligne 66-72)

- Ken el frontend Flask (port 5000) yab3ath requête, CORS yautorisih. Kol domaine ekhir → refus.

3. Enregistrement des Routes (Lignes 147-160)

- Y7ot kol router fi l'application b [app.include_router\(...\)](#). Famma 13 routers majeurs.

[FILE] app/db.py — Connexion MongoDB

Chnowa ya3mel : Yconnecti 3la MongoDB w ye3ref el 8 collections :

Collection	Rôle
users	Les utilisateurs (nom, email, CIN, mot de passe haché)
accounts	Les comptes bancaires (numéro, solde, PIN haché, carte)
transactions	Toutes les opérations (virements, paiements, retraits, dépôts)
otp_codes	Les codes OTP temporaires (6 chiffres, expiration 10 min)
beneficiaries	Les bénéficiaires enregistrés par l'utilisateur
activity_logs	Journal d'activité (login, virement, changement paramètres)
support_messages	Messages de support (réclamations utilisateur → admin)
budgets	Les budgets mensuels définis par catégorie
appointments	Les rendez-vous réservés avec un conseiller

[FILE] app/rate_limiter.py — SlowAPI

Chnowa ya3mel : Y7addad la vitesse mta3 les requêtes par IP.

- Ysta3mel bibliothèque SlowAPI (wrapper de [limits](#) pour FastAPI)
- `key_func=get_remote_address` → Ye3ref el IP mel requête w ya3mel le comptage 3liha
- Ba3dha, kol route ysettilha son propre plafond b `@limiter.limit("5/minute")` etc.

[ROUTE] PARTIE 2 : LES ROUTES API (Dossier `app/routes/`)

[FILE] routes/auth.py — Authentication & Login

Prefix : `/auth` | Chnowa ya3mel : Kol chay li ye5oss el connexion.

Les Endpoints :

Endpoint	Méthode	Rôle
<code>/auth/login</code>	POST	Étape 1 du Login : Vérifie email+password → Génère OTP → Envoie par email
<code>/auth/verify-2fa</code>	POST	Étape 2 du Login : Vérifie le code OTP → Émet le token JWT
<code>/auth/login/biometric</code>	POST	Login par Face ID (reconnaissance faciale)
<code>/auth/forgot-password</code>	POST	Envoie un code de réinitialisation par email
<code>/auth/reset-password</code>	POST	Réinitialise le mot de passe avec le code reçu

Détails Importants :

Le Login en 2 étapes (2FA) :

1. L'utilisateur envoie [email + password](#)
2. Ken valide → le backend ma yab3athech el token direct ! Il génère un OTP 6 chiffres (aléatoire b `random.randint(100000, 999999)`) et l'envoie par email
3. L'utilisateur doit saisir ce code sur [/auth/verify-2fa](#)
4. Ken le code correct et non expiré → le backend émet enfin un JWT (`access_token`)

Protection Brute Force (Lignes 29-51) :

- Ken l'utilisateur fa9ed 3 fois → le compte yet-locki pendant 1 minute (`locked_until`)
- Ken réussi → le compteur yet-resetti à 0

Login Biométrique / Face ID (Lignes 117-186) :

- 2 modes :
- Mode 1 : L'utilisateur donne son email + descripteur facial → on compare avec le descripteur stocké
- Mode 2 : "One-Click Face ID" → on cherche parmi tous les utilisateurs ayant FaceID activé → el plus proche
- Distance euclidienne < 0.7 → Match positif (le visage correspond)

[FILE] routes/user.py — Gestion des Utilisateurs

Prefix : [/users](#) | Chnowa ya3mel : Inscription, profil, paramètres.

Endpoint	Méthode	Rôle
/users/	POST	Inscription (KYC complet : username, lastname, email, CIN, phone, password, OTP)
/users/me	GET	Récupérer le profil de l'utilisateur connecté
/users/me/security	PUT	Modifier username, lastname, ou mot de passe (exige password actuel + OTP)
/users/me/contact	PUT	Modifier email ou téléphone (exige password + OTP)
/users/me/delete	POST	Supprimer le compte et toutes les données associées (cascade)
/users/me/biometric/register	POST	Activer Face ID (enregistrer le descripteur facial)
/users/me/biometric	DELETE	Désactiver Face ID

Détails Sécurité :

Validation du mot de passe (Ligne 22-27) : Regex stricte qui exige :

- Au moins 1 majuscule, 1 minuscule, 1 chiffre, 1 symbole (`@$!%*?&`)
- Entre 6 et 20 caractères

Vérification KYC à l'inscription : CIN doit être exactement 8 chiffres (`^d{8}$`), phone format international, email validé par EmailStr (Pydantic).

Suppression cascade (Lignes 163-167) : Quand l'utilisateur supprime son profil → on supprime aussi ses bénéficiaires, transactions, comptes dans cet ordre.

Vérification d'unicité faciale (Lignes 226-242) : Avant d'enregistrer un visage, on vérifie qu'aucun autre utilisateur n'a un descripteur trop proche (distance < 0.7), pour empêcher qu'un même visage soit lié à 2 comptes.

[FILE] routes/account.py — LE PLUS GROS FICHIER (1418 lignes)

Prefix : `/accounts` | Chnowa ya3mel : Kol les opérations bancaires.

Les Endpoints Principaux :

Endpoint	Méthode	Rôle
<code>/accounts/</code>	POST	Créer un compte (génère numéro 10 chiffres + carte Visa + PIN haché + envoi email)
<code>/accounts/</code>	GET	Lister mes comptes (avec masquage du numéro de carte : <code>4**1234</code>)
<code>/accounts/{num}</code>	GET	Détails d'un compte spécifique
<code>/accounts/deposit</code>	POST	Dépôt d'argent (exige PIN + OTP)
<code>/accounts/withdraw</code>	POST	Retrait (vérifie plafond ATM, carte active, pas expirée, options de retrait)
<code>/accounts/transfer</code>	POST	Virement (vérifie solde suffisant, PIN, OTP, catégorie)
<code>/accounts/payment</code>	POST	Paiement commerçant (vérifie plafond online, option internet, Face ID optionnel)
<code>/accounts/bill-payment</code>	POST	Paiement de factures (eau, électricité, internet, téléphone, gaz)
<code>/accounts/phone-recharge</code>	POST	Recharge téléphonique (Ooredoo, Orange, Tunisie Telecom)
<code>/accounts/checkbook-request</code>	POST	Demande de chéquier (25 ou 50 feuilles)
<code>/accounts/card/limits</code>	PUT	Modifier les plafonds de paiement et retrait (réservé aux abonnés Prime)
<code>/accounts/card/options</code>	PUT	Activer/désactiver les options carte (sans contact, internet, étranger)
<code>/accounts/card/subscription</code>	PUT	Changer l'abonnement carte (Standard, Prime, Business)
<code>/accounts/card/status</code>	PUT	Activer/Désactiver la carte
<code>/accounts/card/renew</code>	POST	Renouveler la carte (nouveau numéro, nouvelle expiration)
<code>/accounts/{num}/transactions</code>	GET	Historique des transactions pour un compte
<code>/accounts/{num}/statement</code>	GET	Relevé PDF téléchargeable (généré avec fpdf)
<code>/accounts/delete</code>	POST	Clôturer un compte

Détails Importants :

Masquage des données bancaires (Ligne 162-164) :

```
# Carte 4539876512341234 → 4*****1234
def mask_card(card_num): return f"{card_num[0]}*****{card_num[-4:]}"
```

Conformité PCI-DSS : on ne montre jamais le numéro complet.

Vérification du PIN (Lignes 183-210) :

- Le PIN est haché (avec Werkzeug [generate_password_hash](#) / [check_password_hash](#))
- 3 échecs → Verrouillage 15 minutes du compte
- Réussite → Réinitialisation du compteur

Vérification de carte expirée (Lignes 212-226) :

- Avant chaque opération : on vérifie que la carte n'est pas expirée (format [MM/YY](#))

Paiement avec Face ID (dans Payment) :

- Si le champ [face_descriptor](#) est fourni → vérification faciale supplémentaire en plus du PIN+OTP

Email de création de compte (Lignes 232-290) :

- Quand un compte est créé → email HTML avec numéro de compte, PIN, et éventuellement les infos carte Visa

[FILE] routes/email_verification.py — Service OTP

Prefix : [/verification](#)

Endpoint	Rôle
/verification/request-otp	Envoyer un code OTP pour l'inscription (3/minute max)
/verification/request-auth-otp	Envoyer un code OTP pour une action authentifiée (virement, paramètres...)

[send_otp_email\(\)](#) : Envoie un email HTML stylisé via SMTP Gmail (SSL port 465) avec le code à 6 chiffres.

- Si Gmail n'est pas configuré → fallback console (affiche le code dans les logs Docker).

[send_admin_action_email\(\)](#) : Envoie une notification au client quand l'admin modifie son statut (blocage, réactivation, etc.)

[FILE] routes/beneficiary.py — Bénéficiaires + CoP

Prefix : [/beneficiaries](#)

Rôle	
GET	Lister mes bénéficiaires
POST	Ajouter un bénéficiaire (exige password + OTP)

La fonctionnalité CoP (Confirmation of Payee) :

Hedhi feature de sécurité bancaire avancée :

1. Ken le compte bénéficiaire est interne (existe fi notre système) :

- On vérifie que le nom fourni correspond au vrai titulaire du compte
- Si ça match → "Bénéficiaire interne ajouté (Vérifié [OK])"
- Si ça match pas → "CoP échouée : le nom ne correspond pas" [FAIL]

2. Ken le compte est externe (pas dans notre banque) :

- On l'accepte mais on marque "Non vérifié [WARN]"

[FILE] routes/budget.py — Budgets Mensuels

Prefix : /budgets

Endpoint	Rôle
POST /budgets/	Définir un budget mensuel pour une catégorie
GET /budgets/{account_number}	Voir tous les budgets + dépenses réelles du mois
DELETE /budgets/	Supprimer un budget

Les 8 catégories : [alimentation](#), [transport](#), [logement](#), [sante](#), [loisirs](#), [shopping](#), [factures](#), [autres](#)

Calcul en temps réel : Le GET récupère toutes les transactions du mois, les classe par catégorie, et calcule :

- Montant dépensé vs. limite définie
- Pourcentage d'utilisation
- Flag [exceeded](#) (dépassé) et [alert](#) ($\geq 80\%$)

[FILE] routes/financial_hub.py — Hub Financier

Prefix : /financial-hub

Endpoint	Rôle
GET /financial-hub/advice/{account}	Conseils financiers IA (Gemini analyse tes budgets et te donne 3 tips)
POST /financial-hub/appointments	Prendre rendez-vous avec un conseiller
GET /financial-hub/appointments	Voir mes rendez-vous
(Admin) GET/POST/PUT/DELETE	Gérer tous les rendez-vous (voir, replanifier, annuler)

Conseils IA : Le système récupère tes budgets et transactions du mois, construit un prompt pour Gemini, et retourne 3 conseils personnalisés. Si Gemini indisponible → conseils génériques en fallback.

[FILE] routes/support.py — Chatbot IA + Messagerie

Prefix : /support

Endpoint	Rôle
POST /support/chat	Chatbot IA — Pose une question, reçois une réponse
POST /support/messages/send	Envoyer un message au conseiller (réclamation structurée)
GET /support/messages/history	Voir l'historique de mes messages

Architecture du Chatbot :

1. Règles déterministes d'abord : Si le message contient "solde", "bloquer", "prime", etc. → réponse codée en dur (fiable et instantanée)
2. IA Gemini en fallback : Si aucune règle ne matche → envoi à Gemini 2.0 Flash Lite avec un system prompt strictement cadré (jamais d'infos financières fausses)

[FILE] routes/admin.py — Dashboard Administrateur

Prefix : /admin

Endpoint	Rôle
GET /admin/stats	Statistiques globales (users, comptes, transactions, volume, sécurité)
GET /admin/users	Liste de tous les utilisateurs
GET /admin/users/{id}	Détails d'un utilisateur + ses comptes + ses transactions
PUT /admin/users/{id}/status	Bloquer/Débloquer un utilisateur (exige OTP admin + envoie email au client)
GET /admin/accounts	Liste de tous les comptes bancaires
PUT /admin/accounts/{num}/status	Suspendre/Réactiver une carte (exige OTP admin + email client)
GET /admin/transactions	Registre complet des transactions (les 500 dernières)
GET /admin/activities	Journal d'activité global
GET /admin/messages	Tous les messages de support (réclamations clients)
PUT /admin/messages/{id}/resolve	Marquer un message comme résolu
POST /admin/block-ip	Bloquer manuellement une IP dans le pare-feu applicatif

Contrôle d'accès : `verify_admin()` vérifie que le token JWT contient `is_admin: true`. Sinon → 403 Accès refusé.

[FILE] routes/alerts.py — Alertes Temps Réel (SSE)

Prefix : /admin

Endpoint	Rôle
GET /admin/alerts/stream	Flux SSE — Connexion persistante, le serveur pousse les alertes automatiquement
GET /admin/alerts/recent	Les 10 dernières alertes (chargement initial)
GET /admin/alerts/correlation	Lancer une corrélation multi-sources à la demande

SSE (Server-Sent Events) :

- Le navigateur de l'admin se connecte une seule fois à [/alerts/stream](#)
- Le serveur envoie un heartbeat toutes les 30 secondes pour garder la connexion vivante
- Dès qu'une nouvelle alerte est ajoutée à [alert_store](#) → elle est poussée vers le navigateur sans recharger la page

[FILE] [routes/activity.py](#) — Journal d'Activité

Prefix : [/activities](#)

Endpoint	Rôle
GET /activities/	Retourne les logs d'activité de l'utilisateur connecté (triés par date décroissante)

Chaque action (virement, modification profil, blocage, etc.) est loguée via [log_activity\(\)](#).

[SECURITY] PARTIE 3 : COUCHE SÉCURITÉ (Dossier ``app/security/``)

[FILE] [security/auth.py](#) — Le Gardien

Chnowa ya3mel : El fichier el akther critique — yet7akkem fi kol les accès.

Les composants :

Fonction	Rôle
hash_password()	Hasher un mot de passe avec bcrypt
verify_password()	Vérifier un mot de passe contre son hash
create_access_token()	Créer un JWT signé en HS256 (token local)
get_keycloak_public_keys()	Télécharger et cacher les clés publiques JWKS de Keycloak
verify_keycloak_token()	Valider un token JWT RS256 de Keycloak avec la clé publique
verify_token()	LA FONCTION CLÉ — appelée par Depends(verify_token) sur presque toutes les routes

`require_scope()`

Vérifier les scopes OAuth2 (read, write, admin)

Comment marche `verify_token()` :

1. Yé9ra el header `Authorization: Bearer <token>` mel requête
2. Y7awel d'abord le valider comme token Keycloak (RS256) via les clés publiques JWKS
3. Ken Keycloak indisponible → fallback sur la validation locale HS256
4. Ken le token valide → retourne le payload (sub=email, id, is_admin, scopes...)
5. Ken le token invalide/expiré → 401 Unauthorized

[FILE] `security/logger.py` — Double Logging

Chnowa ya3mel : Kol action bancaire tet-loggui fi 2 endroits :

1. MongoDB (`activity_logs_collection`) → Pour l'affichage dans le dashboard
2. Fichier JSONL (`data/ai_activity_logs.jsonl`) → Pour l'analyse IA et SIEM (append-only, une ligne JSON par action)

[SOAR/SIEM] PARTIE 4 : MOTEUR SOAR (Dossier ``app/utils/``)

[FILE] `utils/threat_monitor.py` — Le Cerveau SOAR

Chnowa ya3mel : Thread en arrière-plan li ye9ra `security_audit.log` en temps réel w ydetecti les menaces.

Architecture :

```

security_audit.log  ■■→ _monitor_loop()  ■■→ _analyze_line()  ■■→ add_alert()
                    (toutes les 10s)      (regex parsing)      (SSE push)
                    ■
                    ■■■ Brute Force (≥3 x 401) → schedule
_auto_block(30s)    ■■■ Rate Abuse (429)      → schedul
e_auto_block(30s)   ■■■ Forbidden (≥3 x 403)  → schedule
_auto_block(30s)

```

Seuils :

Type d'attaque	Seuil	Poids	Action
Brute Force (401/422 sur <code>/auth/login</code>)	≥ 3 tentatives	w = 1.0	Alerte CRITICAL + Email admin + Email utilisateur ciblé + Blocage auto 30s
Rate Limit Abuse (429)	≥ 1 occurrence	w = 1.5	Alerte HIGH + Email admin + Blocage auto 30s

Accès Interdit (403)	≥ 3 occurrences	w = 1.0	Alerte HIGH + Blocage auto 30s
----------------------	-----------------	---------	--------------------------------

schedule_auto_block() — Le Minuteur SOAR :

- Crée un `threading.Timer(30.0, apply_block)` → Ba3d 30 secondes, ken l'IP mazelet mech bloquée, yblokiha automatiquement
- L'analyste SOC ynajem yannuli mel dashboard avant les 30 secondes (*human-in-the-loop*)

[FILE] utils/security_state.py — Pare-feu Applicatif

Chnowa ya3mel : Maintient un `set()` en mémoire des IPs bloquées.

Fonction	Rôle
<code>block_ip(ip)</code>	Ajoute l'IP au set (sauf IPs internes Docker)
<code>is_ip_blocked(ip)</code>	Vérifie si l'IP est bloquée (appelé par le middleware dans main.py)
<code>unblock_ip(ip)</code>	Retire l'IP du set
<code>get_blocked_ips()</code>	Liste toutes les IPs bloquées

Whitelist : `127.0.0.1`, `0.0.0.0`, et tous les sous-réseaux Docker (`172.x`, `192.168.x`, `10.x`) sont intouchables pour éviter un auto-déni de service.

[FILE] utils/alert_store.py — Bus d'Alertes

Chnowa ya3mel : C'est le pont de communication thread-safe entre le Threat Monitor (thread séparé) et l'endpoint SSE (thread principal FastAPI).

- Utilise une `deque(maxlen=200)` → garde les 200 dernières alertes en mémoire
- `add_alert()` → ajouté par le Threat Monitor ou le Correlator
- `get_alerts_since(last_id)` → lu par le SSE pour pousser les nouvelles alertes

[FILE] utils/security_analyzer.py — Analyse des Logs + IA

Chnowa ya3mel : Parse `security_audit.log` et produit des statistiques de sécurité.

2 modes d'analyse :

1. `rule_based_analysis()` — Analyse heuristique locale (toujours disponible) :
 - Compte les IPs avec le plus d'erreurs 401, 403, 429
 - Identifie les chemins les plus attaqués
 - Génère un résumé structuré
2. `ai_analysis()` — Appel à Gemini 2.0 Flash (quand disponible) :

- Envoie un résumé anonymisé des logs (pas de données personnelles !)
- Demande à l'IA de détecter des patterns et de recommander des actions
- En cas d'indisponibilité → fallback automatique vers l'analyse locale

[FILE] utils/correlator.py — Corrélation Multi-Sources

Chnowa ya3mel : Croise 3 sources de données pour détecter des menaces invisibles si on regarde chaque source toute seule :

Source	Type de données
Source 1 : security_audit.log	IPs avec erreurs HTTP (401, 403, 429)
Source 2 : MongoDB transactions	Comptes avec transactions rejetées récentes
Source 3 : MongoDB users	Utilisateurs actuellement bloqués

Scoring :

- ≥ 3 échecs auth HTTP → +40 points
- ≥ 2 rate limits → +30 points
- ≥ 5 erreurs totales → +20 points
- Transactions rejetées pendant la même fenêtre → +20 points bonus
- Score ≥ 60 → Alerte HIGH ou CRITICAL → Push vers le [alert_store](#)

[FRONTEND] PARTIE 5 : LE FRONTEND (Dossier `bank\_frontend/`)

[FILE] bank_frontend/app.py — Le Serveur Flask (82 Ko)

Sert toutes les 33 pages HTML et fait proxy vers l'API FastAPI backend (<http://backend:8000>).

Les 33 Pages HTML :

Pages Publiques (sans login) :

Page	Rôle
login.html	Page de connexion (email + password)
signup.html	Page d'inscription (KYC complet)

verify_login.html	Saisie du code OTP après login
forgot_password.html	Mot de passe oublié
reset_password.html	Réinitialiser le mot de passe

Pages Client (après login) :

Page	Rôle
dashboard.html	Tableau de bord — Vue d'ensemble des comptes et soldes
account_details.html	Détails d'un compte — Dépôt, Retrait, Virement, Paiement
card_details.html	Ma Carte — Activation, plafonds, options, renouvellement
beneficiaries.html	Gérer mes bénéficiaires (avec CoP)
budget.html	Mes budgets mensuels par catégorie
analytics.html	Graphiques et analytiques de mes dépenses
journal.html	Journal d'activité (toutes les actions loguées)
bills.html	Paiement de factures (eau, électricité, internet...)
services.html	Recharge téléphone, demande chéquier, etc.
qr_payment.html	Paiement par QR Code
rib.html	Mon RIB bancaire
savings.html	Espace Épargne
credit_simulation.html	Simulateur de crédit
credit_request.html	Demande de crédit
scheduled_payments.html	Paiements programmés
advice.html	Conseils financiers IA
agencies.html	Localiser les agences (carte)
messages.html	Messagerie sécurisée avec le conseiller
settings.html	Paramètres du profil (email, tel, password, Face ID)

Pages Admin :

Page	Rôle
admin_dashboard.html	Dashboard SIEM — Stats, alertes SSE, corrélation, blocage IP
admin_users.html	Gestion des utilisateurs (voir, bloquer, débloquent)
admin_user_details.html	Fiche détaillée d'un utilisateur + ses comptes + transactions
admin_accounts.html	Gestion des comptes bancaires
admin_cards.html	Gestion des cartes

admin_messages.html	Réclamations clients
admin_transactions.html	Registre global des transactions

[INFRA] PARTIE 6 : INFRASTRUCTURE (Fichiers racine)

[FILE] nginx.conf — La Passerelle Gateway

4 blocs location :

Location	Cible	Protection
/auth	keycloak:8080	Proxy vers Keycloak (sans mTLS pour le navigateur)
/resources/	keycloak:8080	Fichiers statiques de Keycloak (CSS/JS)
/api/admin/	backend:8000	API Admin (sans mTLS pour le JS du navigateur)
/api/	backend:8000	API Bancaire — mTLS OBLIGATOIRE (\$ssl_client_verify != SUCCESS → 400)
/	frontend:5000	Frontend Flask (tout le reste)

[FILE] docker-compose.yml — Les 10 Services

Service	Image	Rôle	Réseau(x)
backend	banking-api-secure-backend	API FastAPI (port 8000 interne)	backend_tier, data_tier
frontend	banking-api-secure-frontend	Interface Flask (port 5000 interne)	frontend_tier, backend_tier
gateway	nginx:latest	Passerelle NGINX (ports 80, 443 exposés)	frontend_tier, backend_tier
keycloak	keycloak:latest	Serveur IAM (port 8081 exposé)	backend_tier, data_tier
keycloak_db	postgres:15	Base PostgreSQL pour Keycloak	data_tier

elasticsearch	elasticsearch:8.12.0	Moteur d'indexation des logs (port 9200)	data_tier, monitoring_tier
kibana	kibana:8.12.0	Dashboard de visualisation (port 5601)	monitoring_tier
logstash	logstash:8.12.0	Pipeline de parsing des logs (port 5044)	monitoring_tier, data_tier
filebeat	filebeat:8.12.0	Collecteur de logs (lit nginx_logs + security_audit.log)	monitoring_tier, backend_tier
banking_db	mongo:latest	MongoDB pour les données bancaires (port 27017)	(via host.docker.internal)

Les 4 Réseaux :

Réseau	Type	Rôle
frontend_tier	bridge	DMZ — Seul réseau accessible depuis l'extérieur
backend_tier	bridge	Zone applicative privée
data_tier	internal: true	Cœur de données — Aucun accès Internet possible
monitoring_tier	bridge	Supervision ELK isolée

[FILE] Dockerfile.backend — Image Python

```
FROM python:3.10-slim      # Image légère (sans outils superflus → surface d'attaque réduite)
WORKDIR /app              # Répertoire de travail
COPY requirements.txt .    # Copie les dépendances d'abord (optimisation cache Docker)
RUN pip install --no-cache-dir -r requirements.txt # Installation sans cache (image plus petite)
COPY . .                  # Copie le code source
EXPOSE 8000               # Déclare le port (mais ne l'ouvre pas réellement)
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"] # Lance le serveur ASGI
```

3lech [slim](#) w mouch l'image normale ? L'image [slim](#) ma fehach [gcc](#), [wget](#), [curl](#) ni les outils system li un attaquant ynajem ysta3melhom ken youssal yxécuti du code. Hedhi best practice de hardening.

3lech [--no-cache-dir](#) ? Bech el image Docker tkoun aktar sghira w ma testokech les packages téléchargés inutilement.

[FILE] scripts/ — Les Scripts de Test

Script	Rôle
gen_certs.ps1	Génère les certificats mTLS (CA + client + serveur) avec OpenSSL
run_zap_scan.ps1	Lance un scan OWASP ZAP en mode Docker
test_ratelimit.py	Simule une attaque brute force (plein de requêtes rapides)
test_performance.py	Benchmark de latence (100 requêtes séquentielles + 100 concurrentes avec mTLS)

Tawa 3andek el cartographie complète de kol fichier, kol fonction, kol page.

Ma fammech haja fi le projet li ma taarefhé. Rouhi lel soutenance en confiance totale ! ☐